Bicol University
POLANGUI CAMPUS
Centro Occidental Polangui, Albay

IT112 Web System Development

PROJECT PROPOSAL BY Tech-net

Group Members:
Mico Malana
Christian Silo
Jay Lloyd Chicogo
Nelson Andes
Allen Benedict

Submitted to: Guillermo Jr. V. Red

Bicol University
POLANGUI CAMPUS
Centro Occidental Polangui, Albay

```
const express = require('express');    // File is a CommonJS module; it may be converted to an ES module.

const {
  getTasks,
  createTask,
  updateTask,
} = require('../controllers/task.controller');

const taskRouter = express.Router();

const authMiddleware = require('../middlewares/task.middleware');

// GET ALL THE TASKS
taskRouter.get('/tasks', authMiddleware, async (req : Request<{}, any, any, Parse..., res : Response<any, Record<string,>> ) : Promise<Response<any, Recor. => {
  const { id } = req.student;
  try {
    const tasks = await getTasks(id);
    if (!tasks) return res.json({ message: 'Tasks is empty.' });

    res.status(code: 200).json({ tasks });
  } catch (error) {
    res.status(code: 500).json({ message: error.message });
  }
});

// CREATE TASK
taskRouter.post('/task/create', async (req : Request<{}, any, any, Parse..., res : Response<any, Record<string,>> ) : Promise<void> => {
  const task = req.body;

  try {
    const newTask = await createTask(task);
    res.status(code: 201).json({ newTask });
  } catch (error) {
    res.status(code: 500).json({ message: error.message });
  }
});

// UPDATE TASK
taskRouter.put('/task/update/:id', async (req : Request<{ id: string }, any,, res : Response<any, Record<string,>> ) : Promise<void> => {
  const task = req.body;
  try {
    const newTask = await updateTask(req.params.id, task);
    if (!newTask)
      res.status(code: 404).json({ message: 'Task not found or unauthorized' });

    res.status(code: 200).json({ newTask });
  } catch (error) {
    res.status(code: 500).json({ message: error.message });
  }
});

module.exports = taskRouter;
```


```
const express = require('express');    // File is a CommonJS module; it may be converted to an ES module.

const {
  getTasks,
  createTask,
  updateTask,
} = require('../controllers/task.controller');

const taskRouter = express.Router();

const authMiddleware = require('../middlewares/task.middleware');

// GET ALL THE TASKS
taskRouter.get('/tasks', authMiddleware, async (req : Request<{}, any, any, Parse..., res : Response<any, Record<string,>> ) : Promise<Response<any, Recor. => {
  const { id } = req.student;
  try {
    const tasks = await getTasks(id);
    if (!tasks) return res.json({ message: 'Tasks is empty.' });

    res.status(code: 200).json({ tasks });
  } catch (error) {
    res.status(code: 500).json({ message: error.message });
  }
});

// CREATE TASK
taskRouter.post('/task/create', async (req : Request<{}, any, any, Parse..., res : Response<any, Record<string,>> ) : Promise<void> => {
  const task = req.body;

  try {
    const newTask = await createTask(task);
    res.status(code: 201).json({ newTask });
  } catch (error) {
    res.status(code: 500).json({ message: error.message });
  }
});

// UPDATE TASK
taskRouter.put('/task/update/:id', async (req : Request<{ id: string }, any,, res : Response<any, Record<string,>> ) : Promise<void> => {
  const task = req.body;
  try {
    const newTask = await updateTask(req.params.id, task);
    if (!newTask)
      res.status(code: 404).json({ message: 'Task not found or unauthorized' });

    res.status(code: 200).json({ newTask });
  } catch (error) {
    res.status(code: 500).json({ message: error.message });
  }
});

module.exports = taskRouter;
```


```
const express = require('express');    // File is a CommonJS module; it may be converted to an ES module.
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
require('dotenv').config();

const studentRouter = express.Router();

const {
  createStudent,
  findStudent,
  findStudentById,
  getAllStudents,
} = require('../controllers/student.controller');

studentRouter.get('/student', (req : Request<{}, any, any, Parse..., res : Response<any, Record<string,>> ) : void => {    // 'req' is declared but its value is never read.
  res.send({ message: "Welcome to student" });
});

// REGISTER
studentRouter.post('/student/register', async (req : Request<{}, any, any, Parse..., res : Response<any, Record<string,>> ) : Promise<void> => {
  try {
    const { name, email, password } = req.body;

    const hashedPassword = await bcrypt.hash(password, saltOrRounds: 10);
    const student = await createStudent({
      name,
      email,
      password: hashedPassword,
    });

    res.status(code: 201).json({ student });
  } catch (error) {
    res.status(code: 500).json({ message: error.message });
  }
});

// LOGIN
studentRouter.post('/student/login', async (req : Request<{}, any, any, Parse..., res : Response<any, Record<string,>> ) : Promise<Response<any, Recor. => {
  try {
    const { email, password } = req.body;

    const student = await findStudent(email);

    if (!student)
      return res.status(code: 404).json({ message: "Student not found." });

    const isMatch = await bcrypt.compare(password, student.password);

    if (!isMatch)
      return res.status(code: 401).json({ message: "Invalid credentials." });

    const token = jwt.sign({ id: student._id }, process.env.JWT_SECRET, {
      expiresIn: "1h",
    });

    res.status(code: 200).json({ token });
  } catch (error) {
    res.status(code: 500).json({ message: error.message });
  }
```

```
});

// SEARCH BY ID (TESTING PURPOSES)
studentRouter.get('/student/:id', async (req : Request<{ id: string }, any,, res : Response<any, Record<string,>> ) : Promise<void> => {
  try {
    const id = req.params.id;
    const student = await findStudentById(id);

    if (!student) res.status(code: 404).json({ message: "Student not found." });
    res.status(code: 200).json({ student });
  } catch (error) {
    res.status(code: 500).json({ message: error.message });
  }
});

// GET ALL STUDENTS (TESTING PURPOSES)
studentRouter.get("/students", async (req : Request<{}, any, any, Parse..., res : Response<any, Record<string,>> ) : Promise<void> => {    // 'req' is declared but its value is never rea
  try {
    const students = await getAllStudents();
    res.status(code: 200).json({ students });
  } catch (error) {
    res.status(code: 500).json({ message: error.message });
  }
});

module.exports = studentRouter;
```

Backend Route Functionality Overview

This report outlines the implementation of a Node.js REST API using Express Routers for managing students, groups, and tasks.

1. Student Router (studentRouter)

This module handles user authentication (Registration/Login) and student data retrieval.

Route	Method	Description
/student	GET	A landing/test route that returns a welcome message.
/student/register	POST	Registration: Destructures name, email, and password from the request body. It hashes the password using bcrypt before saving the student to the database. nt/login
/student/login	POST	Authentication: Checks if the student uvvUIu nmpares hashed vers pasonfith the hashed version. If successful, it signs a JWT (JSON Web Token) with a 1-hour expiration.
/student/:id	GET	Search by ID: Retrieves a specific student's profile using the ID passed in the URL parameters.
/students	GET	Admin/Testing: Fetches a complete list of all registered students.

2. Group Router (groupRouter)

This module manages the social/collaborative aspect of the application.

- **GET /group**: Returns a status message confirming the user is on a group-related route.
- **POST /group/create**: Accepts a group object from req.body and calls the createGroup controller to persist a new group in the database.

- **POST /group/join**: Processes a membership request (likely containing a studentId and groupId) to link a student to an existing group.

3. Task Router (taskRouter)

This module handles CRUD operations for tasks and utilizes middleware for security.

- **Middleware Integration:** Unlike the other routers, the GET /tasks route uses authMiddleware. This ensures that only authenticated users can view tasks.
- **Functionality:**
  - **Get Tasks:** Retrieves tasks associated with the student ID (extracted from req.student, which is likely populated by the auth middleware).
  - **Create Task:** Takes task details from the request body and creates a new entry.
  - **Update Task:** Uses a URL parameter (:id) to identify a specific task and updates its content/status via req.body.

4. Technical Implementation Notes

Error Handling

Every route is wrapped in a try...catch block.

- **Success:** Returns HTTP 200 (OK) or 201 (Created).
- **Failure:** Returns HTTP 500 (Internal Server Error) with the specific error message as a JSON response.

Security Practices

- **Password Salting:** The system uses bcrypt.hash(password, 10) to ensure passwords are never stored in plain text.
- **Tokenization:** Upon successful login, a JWT is generated. This token is used to authorize subsequent requests in the Task Router.

Identified Linter Warnings

In several screenshots (notably the GET /student and GET /group routes), there is a pink warning:

'req' is declared but its value is never read.

This indicates that while the request object is passed into the callback function, it isn't being used within the logic of those specific test routes. This can be resolved by replacing req with _req to signal it is an intentional unused variable.